

Indices: B+trees

Clase 14

IIC2413 - Sección 3

Prof. Cristian Riveros

Outline

B+ trees

Insertión en B+ trees

Eliminación en B+ trees

Outline

B+trees

Inserción en B+trees

Eliminación en B+trees

Páginas ordenadas y búsqueda binaria

Páginas ordenadas y búsqueda binaria

Ejemplo: búsqueda por rango

```
SELECT *  
FROM   Players  
WHERE  age >= 31
```

Players			
id	name	age	team
10	Lionel Messi	39	Inter Miami
9	Erling Haaland	26	Manchester City
8	Pedro López	23	Barcelona
⋮	⋮	⋮	⋮

Páginas ordenadas y búsqueda binaria

Ejemplo: búsqueda por rango

```
SELECT *  
FROM   Players  
WHERE  age >= 31
```

Players			
id	name	age	team
10	Lionel Messi	39	Inter Miami
9	Erling Haaland	26	Manchester City
8	Pedro López	23	Barcelona
⋮	⋮	⋮	⋮

14* 15* 17* 19* 22* 23* 26* 27* 29* 30* 31* 33* 34* 35* 39* 42*

Páginas ordenadas y búsqueda binaria

Ejemplo: búsqueda por rango

```
SELECT *  
FROM   Players  
WHERE  age >= 31
```

Players			
id	name	age	team
10	Lionel Messi	39	Inter Miami
9	Erling Haaland	26	Manchester City
8	Pedro López	23	Barcelona
⋮	⋮	⋮	⋮

14* 15* 17* 19* 22* 23* 26* 27* 29* 30* 31* 33* 34* 35* 39* 42*

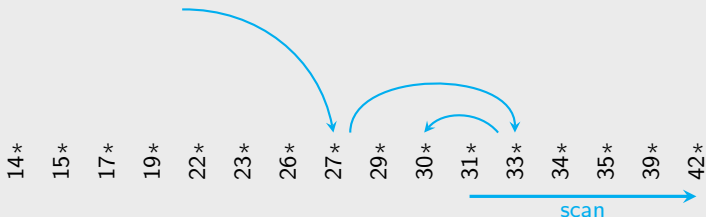


Páginas ordenadas y búsqueda binaria

Ejemplo: búsqueda por rango

```
SELECT *  
FROM   Players  
WHERE  age >= 31
```

Players			
id	name	age	team
10	Lionel Messi	39	Inter Miami
9	Erling Haaland	26	Manchester City
8	Pedro López	23	Barcelona
⋮	⋮	⋮	⋮

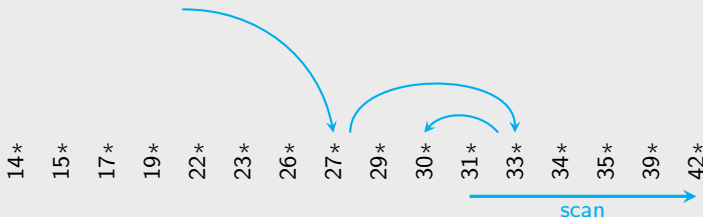


Páginas ordenadas y búsqueda binaria

Ejemplo: búsqueda por rango

```
SELECT *  
FROM   Players  
WHERE  age >= 31
```

Players			
id	name	age	team
10	Lionel Messi	39	Inter Miami
9	Erling Haaland	26	Manchester City
8	Pedro López	23	Barcelona
⋮	⋮	⋮	⋮



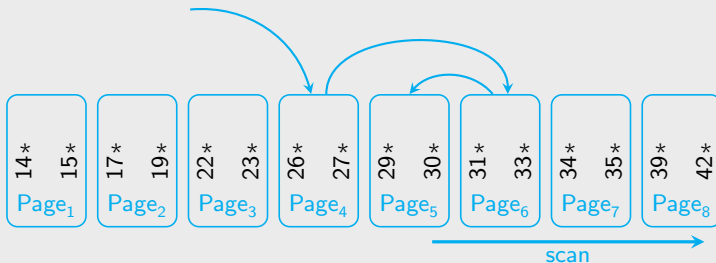
¿cuánto **tiempo** nos tardamos en esta búsqueda?

Páginas ordenadas y búsqueda binaria

Ejemplo: búsqueda por rango

```
SELECT *  
FROM   Players  
WHERE  age >= 31
```

Players			
id	name	age	team
10	Lionel Messi	39	Inter Miami
9	Erling Haaland	26	Manchester City
8	Pedro López	23	Barcelona
⋮	⋮	⋮	⋮

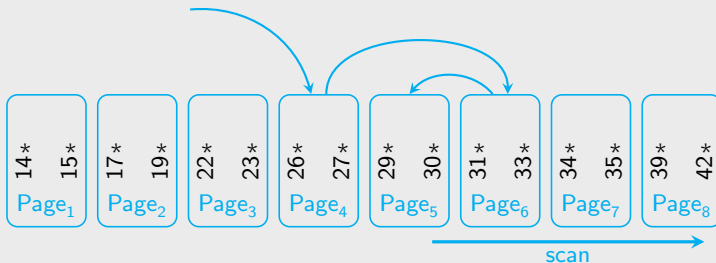


Páginas ordenadas y búsqueda binaria

Ejemplo: búsqueda por rango

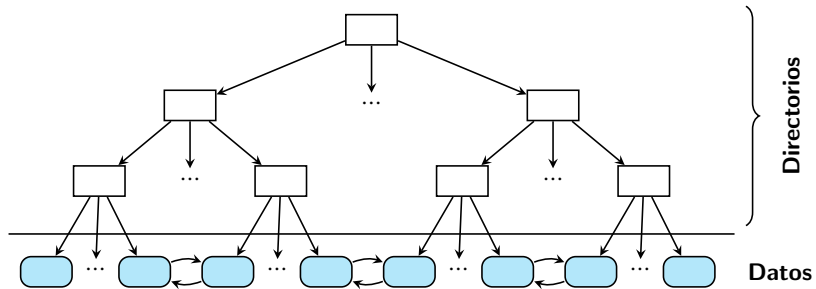
```
SELECT *  
FROM   Players  
WHERE  age >= 31
```

Players			
id	name	age	team
10	Lionel Messi	39	Inter Miami
9	Erling Haaland	26	Manchester City
8	Pedro López	23	Barcelona
⋮	⋮	⋮	⋮

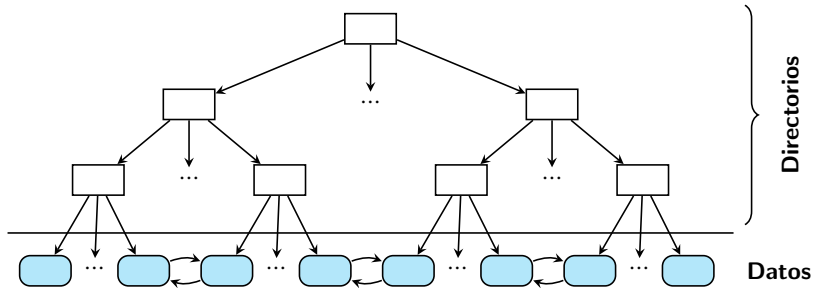


¿cuál es el **problema** con este índice?

B+tree

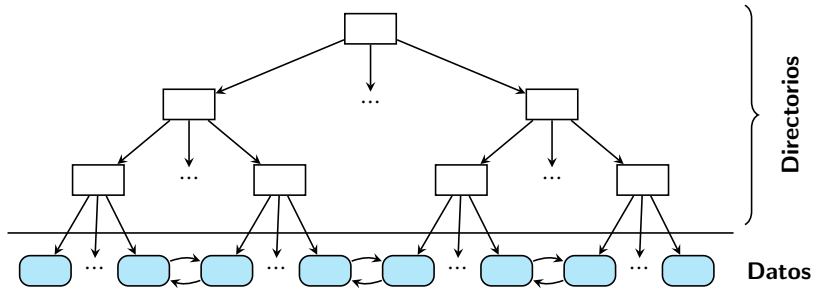


B+tree



"B+trees are by far the most important access path structure in databases and file systems", Gray y Reuter (1993).

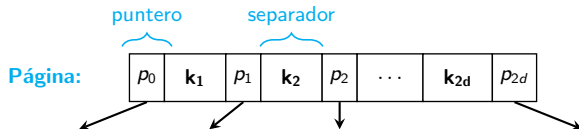
B+tree



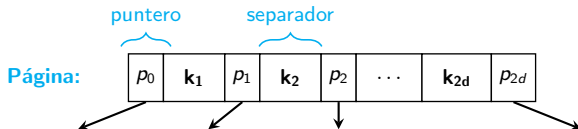
"B+trees are by far the most important access path structure in databases and file systems", Gray y Reuter (1993).

En la **mayoría** de los sistemas de bases de datos,
CREATE INDEX crea un B+tree **por default**

B+tree: estructura de un directorio

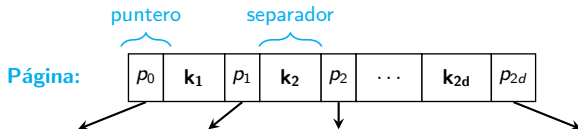


B+tree: estructura de un directorio



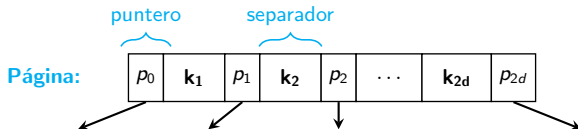
- d es el **orden** del B+tree: $d \approx \text{tamaño de página}/2$.

B+tree: estructura de un directorio



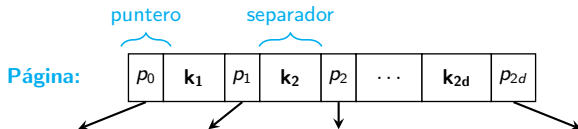
- d es el **orden** del B+tree: $d \approx \text{tamaño de página}/2$.
- Los valores k^* apuntados por p_i satisfacen: $k_i \leq k^* < k_{i+1}$

B+tree: estructura de un directorio



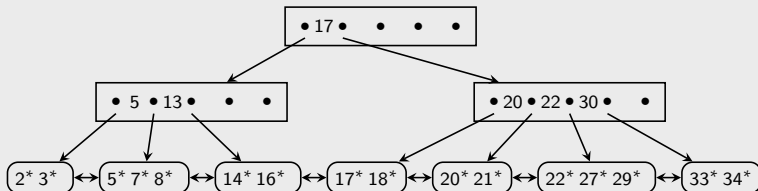
- d es el **orden** del B+tree: $d \approx \text{tamaño de página}/2$.
- Los valores k^* apuntados por p_i satisfacen: $k_i \leq k^* < k_{i+1}$
- Hojas están **ordenados** y sus páginas están **doblemente ligadas**.

B+tree: estructura de un directorio

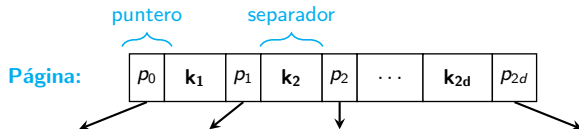


- d es el **orden** del B+tree: $d \approx \text{tamaño de página}/2$.
- Los valores k^* apuntados por p_i satisfacen: $k_i \leq k^* < k_{i+1}$
- Hojas están **ordenados** y sus páginas están **doblemente ligadas**.

Ejemplo: B+tree de orden $d = 2$

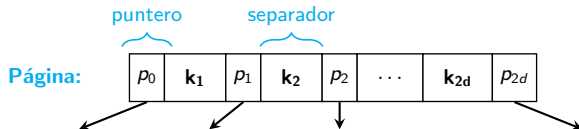


¿cómo buscamos un dato en un B+tree?



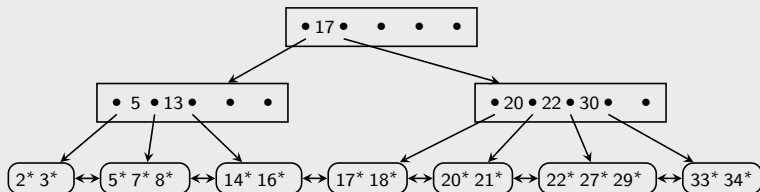
- d es el **orden** del B+tree: $d \approx \text{tamaño de página}/2$.
- Los valores k^* apuntados por p_i satisfacen: $k_i \leq k^* < k_{i+1}$
- Hojas están **ordenados** y sus páginas están **doblemente ligadas**.

¿cómo buscamos un dato en un B+tree?

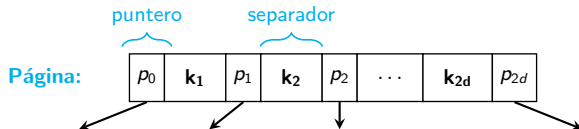


- d es el **orden** del B+tree: $d \approx \text{tamaño de página}/2$.
- Los valores k^* apuntados por p_i satisfacen: $k_i \leq k^* < k_{i+1}$
- Hojas están **ordenados** y sus páginas están **doblemente ligadas**.

Ejemplo: ¿cómo encontramos los valores mayores a 27^* ?

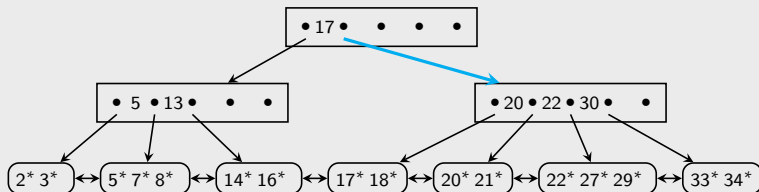


¿cómo buscamos un dato en un B+tree?

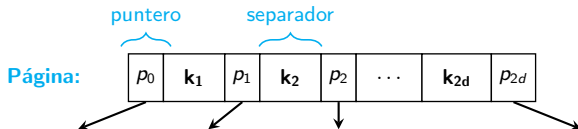


- d es el **orden** del B+tree: $d \approx \text{tamaño de página}/2$.
- Los valores k^* apuntados por p_i satisfacen: $k_i \leq k^* < k_{i+1}$
- Hojas están **ordenados** y sus páginas están **doblemente ligadas**.

Ejemplo: ¿cómo encontramos los valores mayores a 27^* ?

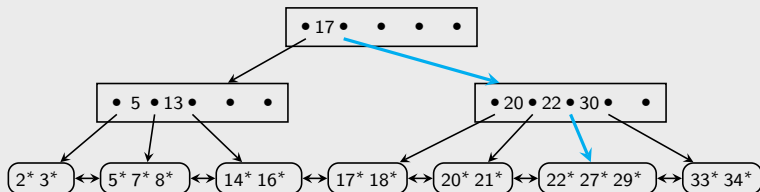


¿cómo buscamos un dato en un B+tree?

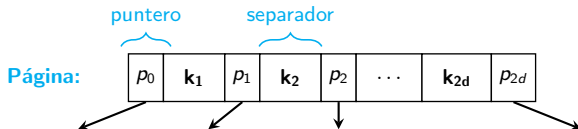


- d es el **orden** del B+tree: $d \approx \text{tamaño de página}/2$.
- Los valores k^* apuntados por p_i satisfacen: $k_i \leq k^* < k_{i+1}$
- Hojas están **ordenados** y sus páginas están **doblemente ligadas**.

Ejemplo: ¿cómo encontramos los valores mayores a 27^* ?

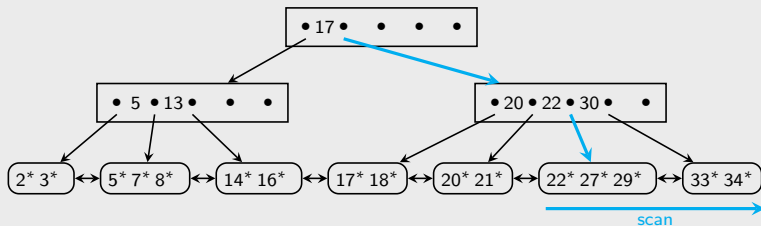


¿cómo buscamos un dato en un B+tree?



- d es el **orden** del B+tree: $d \approx \text{tamaño de página}/2$.
- Los valores k^* apuntados por p_i satisfacen: $k_i \leq k^* < k_{i+1}$
- Hojas están **ordenados** y sus páginas están **doblemente ligadas**.

Ejemplo: ¿cómo encontramos los valores mayores a 27^* ?



¿cómo buscamos un dato en un B+tree?

Algoritmo para encontrar un dato/página

input : Un valor k^* y una página P del índice.

output: Una página de datos que puede contener a k^* .

1 **Function** busquedaEnArbol (k, P)

¿cómo buscamos un dato en un B+tree?

Algoritmo para encontrar un dato/página

input : Un valor k^* y una página P del índice.

output: Una página de datos que puede contener a k^* .

```
1 Function busquedaEnArbol ( $k, P$ )  
2   if  $P$  es una página de datos then  
3     return  $P$ 
```

¿cómo buscamos un dato en un B+tree?

Algoritmo para encontrar un dato/página

input : Un valor k^* y una página P del índice.

output: Una página de datos que puede contener a k^* .

```
1 Function busquedaEnArbol ( $k, P$ )  
2   if  $P$  es una página de datos then  
3     return  $P$   
4   let  $P = p_0 k_1 p_1 \dots k_n p_n$   
5   switch  $k$  do
```

¿cómo buscamos un dato en un B+tree?

Algoritmo para encontrar un dato/página

input : Un valor k^* y una página P del índice.

output: Una página de datos que puede contener a k^* .

```
1 Function busquedaEnArbol ( $k, P$ )  
2   if  $P$  es una página de datos then  
3     return  $P$   
4   let  $P = p_0 k_1 p_1 \dots k_n p_n$   
5   switch  $k$  do  
6     case  $k < k_1$  do  
7       return busquedaEnArbol ( $k, p_0$ )
```

¿cómo buscamos un dato en un B+tree?

Algoritmo para encontrar un dato/página

input : Un valor k^* y una página P del índice.

output: Una página de datos que puede contener a k^* .

```
1 Function busquedaEnArbol ( $k, P$ )  
2   if  $P$  es una página de datos then  
3     return  $P$   
4   let  $P = p_0 k_1 p_1 \dots k_n p_n$   
5   switch  $k$  do  
6     case  $k < k_1$  do  
7       return busquedaEnArbol ( $k, p_0$ )  
8     case  $k_i \leq k < k_{i+1}$  do  
9       return busquedaEnArbol ( $k, p_i$ )
```

¿cómo buscamos un dato en un B+tree?

Algoritmo para encontrar un dato/página

input : Un valor k^* y una página P del índice.

output: Una página de datos que puede contener a k^* .

```
1 Function busquedaEnArbol ( $k, P$ )  
2   if  $P$  es una página de datos then  
3     return  $P$   
4   let  $P = p_0 k_1 p_1 \dots k_n p_n$   
5   switch  $k$  do  
6     case  $k < k_1$  do  
7       return busquedaEnArbol ( $k, p_0$ )  
8     case  $k_i \leq k < k_{i+1}$  do  
9       return busquedaEnArbol ( $k, p_i$ )  
10    case  $k_n \leq k$  do  
11      return busquedaEnArbol ( $k, p_n$ )
```

¿cómo respondemos una consulta con un B+tree?

```
SELECT  *  
FROM    tabla  
WHERE   v1 <= atributo AND atributo <= v2
```

¿cómo respondemos una consulta con un B+tree?

```
SELECT  *  
FROM    tabla  
WHERE   v1 <= atributo AND atributo <= v2
```

1. Llamar $P := \text{busquedaEnArbol}(v1, \text{root})$.

¿cómo respondemos una consulta con un B+tree?

```
SELECT  *  
FROM    tabla  
WHERE   v1 <= atributo AND atributo <= v2
```

1. Llamar $P := \text{busquedaEnArbol}(v1, \text{root})$.
2. **Búsqueda binaria** del mayor elemento k^* en P tal que $k^* \leq v1$

¿cómo respondemos una consulta con un B+tree?

```
SELECT  *  
FROM    tabla  
WHERE   v1 <= atributo AND atributo <= v2
```

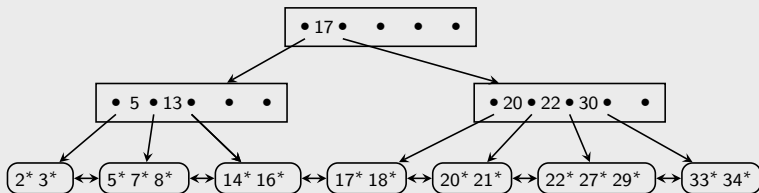
1. Llamar $P := \text{busquedaEnArbol}(v1, \text{root})$.
2. **Búsqueda binaria** del mayor elemento k^* en P tal que $k^* \leq v1$
3. Hacer **scan** desde k^* sobre todos los valores menores o iguales a $v2$.

¿cómo respondemos una consulta con un B+tree?

```
SELECT  *  
FROM    tabla  
WHERE   v1 <= atributo AND atributo <= v2
```

1. Llamar $P := \text{busquedaEnArbol}(v1, \text{root})$.
2. **Búsqueda binaria** del mayor elemento k^* en P tal que $k^* \leq v1$
3. Hacer **scan** desde k^* sobre todos los valores menores o iguales a $v2$.

Ejemplo: `SELECT * FROM R WHERE 15 <= A AND A <= 24`

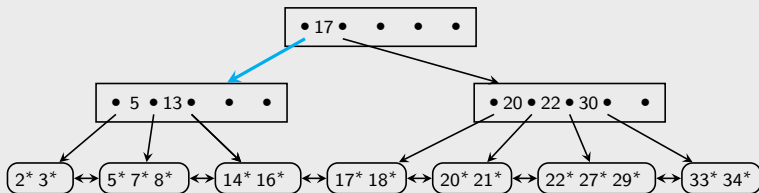


¿cómo respondemos una consulta con un B+tree?

```
SELECT  *  
FROM    tabla  
WHERE   v1 <= atributo AND atributo <= v2
```

1. Llamar $P := \text{busquedaEnArbol}(v1, \text{root})$.
2. **Búsqueda binaria** del mayor elemento k^* en P tal que $k^* \leq v1$
3. Hacer **scan** desde k^* sobre todos los valores menores o iguales a $v2$.

Ejemplo: `SELECT * FROM R WHERE 15 <= A AND A <= 24`

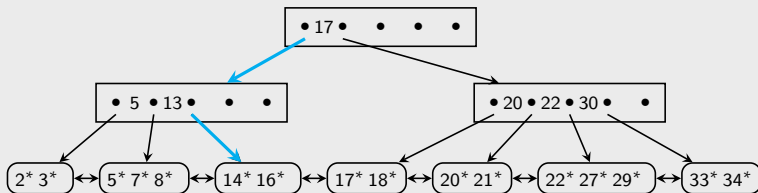


¿cómo respondemos una consulta con un B+tree?

```
SELECT  *  
FROM    tabla  
WHERE   v1 <= atributo AND atributo <= v2
```

1. Llamar $P := \text{busquedaEnArbol}(v1, \text{root})$.
2. **Búsqueda binaria** del mayor elemento k^* en P tal que $k^* \leq v1$
3. Hacer **scan** desde k^* sobre todos los valores menores o iguales a $v2$.

Ejemplo: `SELECT * FROM R WHERE 15 <= A AND A <= 24`

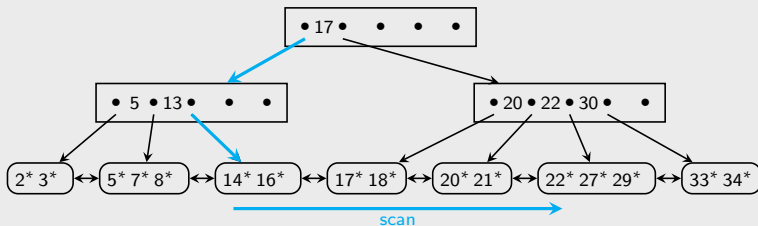


¿cómo respondemos una consulta con un B+tree?

```
SELECT  *  
FROM    tabla  
WHERE   v1 <= atributo AND atributo <= v2
```

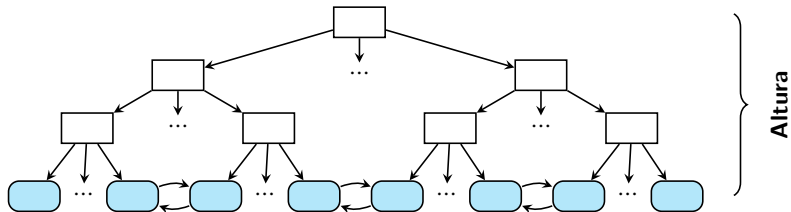
1. Llamar $P := \text{busquedaEnArbol}(v1, \text{root})$.
2. **Búsqueda binaria** del mayor elemento k^* en P tal que $k^* \leq v1$
3. Hacer **scan** desde k^* sobre todos los valores menores o iguales a $v2$.

Ejemplo: `SELECT * FROM R WHERE 15 <= A AND A <= 24`

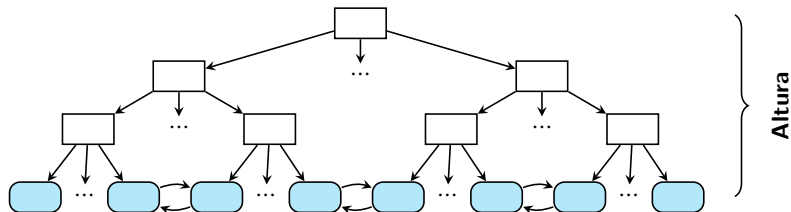


¿cuál es el tiempo de búsqueda en un B+tree?

¿cuál es el tiempo de búsqueda en un B+tree?



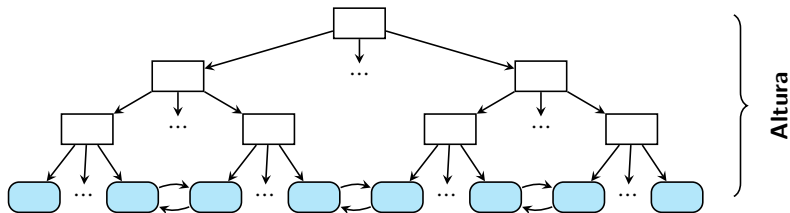
¿cuál es el tiempo de búsqueda en un B+tree?



Definición

- La **altura** de un **nodo** es el número de aristas del camino más largo desde el nodo hasta alguna hoja.

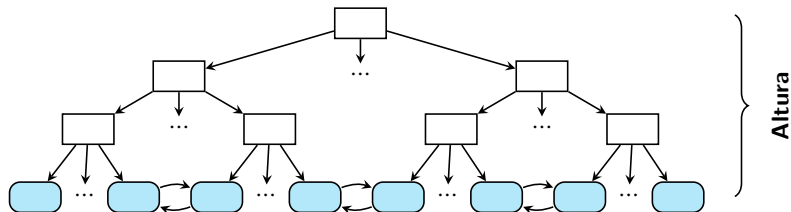
¿cuál es el tiempo de búsqueda en un B+tree?



Definición

- La **altura** de un **nodo** es el número de aristas del camino más largo desde el nodo hasta alguna hoja.
- La **altura del árbol** se define como la altura de la **raíz**.

¿cuál es el tiempo de búsqueda en un B+tree?

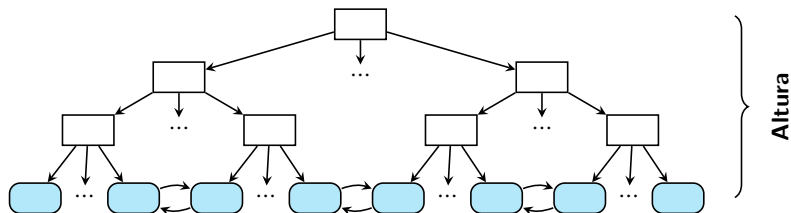


Definición

- La **altura** de un **nodo** es el número de aristas del camino más largo desde el nodo hasta alguna hoja.
- La **altura del árbol** se define como la altura de la **raíz**.

tiempo de
una búsqueda = **altura** B+tree

¿cuál es el tiempo de búsqueda en un B+tree?



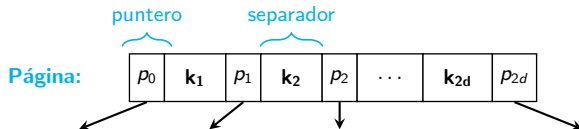
Definición

- La **altura** de un **nodo** es el número de aristas del camino más largo desde el nodo hasta alguna hoja.
- La **altura del árbol** se define como la altura de la **raíz**.

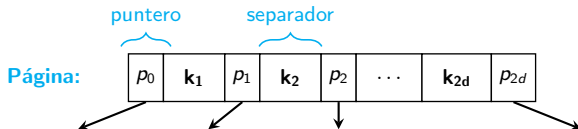
tiempo de
una búsqueda = **altura** B+tree

¿cómo nos aseguramos que la **altura** del B+tree sea **baja**?

Condición de balance de un B+tree



Condición de balance de un B+tree

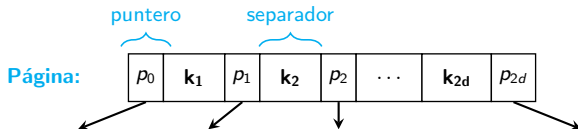


Regla de altura de hijos

Para todo nodo n y sus hijos $n_1, \dots, n_\ell$ se cumple que:

$$\text{altura } n_1 = \text{altura } n_2 = \dots = \text{altura } n_\ell$$

Condición de balance de un B+tree



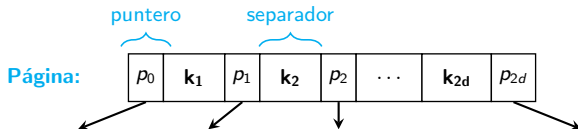
Regla de altura de hijos

Para todo nodo n y sus hijos $n_1, \dots, n_\ell$ se cumple que:

$$\text{altura } n_1 = \text{altura } n_2 = \dots = \text{altura } n_\ell$$

Todos los hijos están a la **misma altura**

Condición de balance de un B+tree



Regla de altura de hijos

Para todo nodo n y sus hijos $n_1, \dots, n_\ell$ se cumple que:

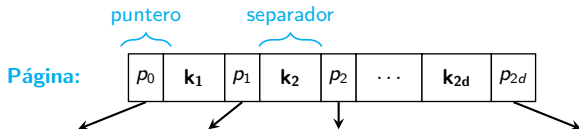
$$\text{altura } n_1 = \text{altura } n_2 = \dots = \text{altura } n_\ell$$

Regla de ocupación mínima

Si d es el orden de un B+tree:

- Toda **hoja** tiene **al menos d datos** y a los más $2d$ datos.

Condición de balance de un B+tree



Regla de altura de hijos

Para todo nodo n y sus hijos $n_1, \dots, n_\ell$ se cumple que:

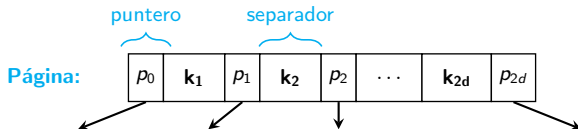
$$\text{altura } n_1 = \text{altura } n_2 = \dots = \text{altura } n_\ell$$

Regla de ocupación mínima

Si d es el orden de un B+tree:

- Toda **hoja** tiene **al menos d datos** y a los más $2d$ datos.
- Todo **nodo** (excepto la raíz) tiene **al menos d punteros** usados.

Condición de balance de un B+tree



Regla de altura de hijos

Para todo nodo n y sus hijos $n_1, \dots, n_\ell$ se cumple que:

$$\text{altura } n_1 = \text{altura } n_2 = \dots = \text{altura } n_\ell$$

Regla de ocupación mínima

Si d es el orden de un B+tree:

- Toda **hoja** tiene **al menos d datos** y a los más $2d$ datos.
- Todo **nodo** (excepto la raíz) tiene **al menos d punteros** usados.

Todos los nodos están **medio llenos**

Condición de balance de un B+tree

Regla de altura de hijos

Para todo nodo n y sus hijos $n_1, \dots, n_\ell$ se cumple que:

$$\text{altura } n_1 = \text{altura } n_2 = \dots = \text{altura } n_\ell$$

Regla de ocupación mínima

Si d es el orden de un B+tree:

- Toda **hoja** tiene **al menos d datos** y a los más $2d$ datos.
- Todo **nodo** (excepto la raíz) tiene **al menos d punteros** usados.

Condición de balance de un B+tree

Regla de altura de hijos

Para todo nodo n y sus hijos $n_1, \dots, n_\ell$ se cumple que:

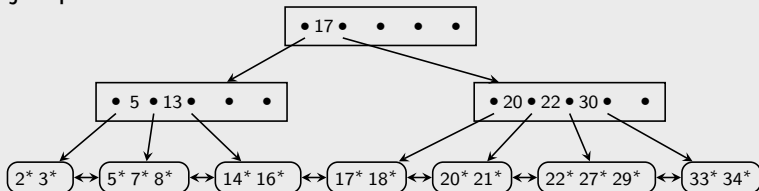
$$\text{altura } n_1 = \text{altura } n_2 = \dots = \text{altura } n_\ell$$

Regla de ocupación mínima

Si d es el orden de un B+tree:

- Toda **hoja** tiene **al menos d datos** y a los más $2d$ datos.
- Todo **nodo** (excepto la raíz) tiene **al menos d punteros** usados.

Ejemplo: $d = 2$



Condición de balance de un B+tree

Teorema

Para todo B+tree de orden d y con n datos se cumple que:

$$\text{altura B+tree} \leq \mathcal{O}(\log_d(n))$$

Condición de balance de un B+tree

Teorema

Para todo B+tree de orden d y con n datos se cumple que:

$$\text{altura B+tree} \leq \mathcal{O}(\log_d(n))$$

Demostración (idea)

Condición de balance de un B+tree

Teorema

Para todo B+tree de orden d y con n datos se cumple que:

$$\text{altura B+tree} \leq \mathcal{O}(\log_d(n))$$

Demostración (idea)

Sea H la altura del B+tree.

Condición de balance de un B+tree

Teorema

Para todo B+tree de orden d y con n datos se cumple que:

$$\text{altura B+tree} \leq \mathcal{O}(\log_d(n))$$

Demostración (idea)

Sea H la altura del B+tree.

- Cada hoja tiene al menos d datos.

(¿por qué?)

Condición de balance de un B+tree

Teorema

Para todo B+tree de orden d y con n datos se cumple que:

$$\text{altura B+tree} \leq \mathcal{O}(\log_d(n))$$

Demostración (idea)

Sea H la altura del B+tree.

- Cada hoja tiene al menos d datos. (¿por qué?)
- Cada nodo de altura h tiene al menos d^h datos. (¿por qué?)

Condición de balance de un B+tree

Teorema

Para todo B+tree de orden d y con n datos se cumple que:

$$\text{altura B+tree} \leq \mathcal{O}(\log_d(n))$$

Demostración (idea)

Sea H la altura del B+tree.

- Cada hoja tiene al menos d datos. (¿por qué?)
- Cada nodo de altura h tiene al menos d^h datos. (¿por qué?)
- La raíz tiene al menos $2 \cdot d^{H-1}$ datos. (¿por qué?)

Condición de balance de un B+tree

Teorema

Para todo B+tree de orden d y con n datos se cumple que:

$$\text{altura B+tree} \leq \mathcal{O}(\log_d(n))$$

Demostración (idea)

Sea H la altura del B+tree.

- Cada hoja tiene al menos d datos. (¿por qué?)
- Cada nodo de altura h tiene al menos d^h datos. (¿por qué?)
- La raíz tiene al menos $2 \cdot d^{H-1}$ datos. (¿por qué?)

Por lo tanto:

$$2 \cdot d^{H-1} \leq n$$

Condición de balance de un B+tree

Teorema

Para todo B+tree de orden d y con n datos se cumple que:

$$\text{altura B+tree} \leq \mathcal{O}(\log_d(n))$$

Demostración (idea)

Sea H la altura del B+tree.

- Cada hoja tiene al menos d datos. (¿por qué?)
- Cada nodo de altura h tiene al menos d^h datos. (¿por qué?)
- La raíz tiene al menos $2 \cdot d^{H-1}$ datos. (¿por qué?)

Por lo tanto:

$$2 \cdot d^{H-1} \leq n$$

Despejando:

$$H \leq \log_d \left(\frac{n}{2} \right) + 1$$



Condición de balance de un B+tree

Teorema

Para todo B+tree de orden d y con n datos se cumple que:

$$\text{altura B+tree} \in \mathcal{O}(\log_d(n))$$

Tiempo de búsqueda en un B+tree

- n : el número de tuplas.
- B : el tamaño de una página.
- $d \approx B/2$

Condición de balance de un B+tree

Teorema

Para todo B+tree de orden d y con n datos se cumple que:

$$\text{altura B+tree} \in \mathcal{O}(\log_d(n))$$

Tiempo de búsqueda en un B+tree

- n : el número de tuplas.
- B : el tamaño de una página.
- $d \approx B/2$

Tiempo de búsqueda: $\mathcal{O}(\log_B(n))$

Outline

B+trees

Inserción en B+trees

Eliminación en B+trees

Insertar un elemento en B+-trees (orden d)

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Inserción en B+-tree

Para **insertar** un valor k :

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Inserción en B+-tree

Para **insertar** un valor k :

1. Llamar $P = \text{busquedaEnArbol}(k, \text{root})$.

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Inserción en B+-tree

Para **insertar** un valor k :

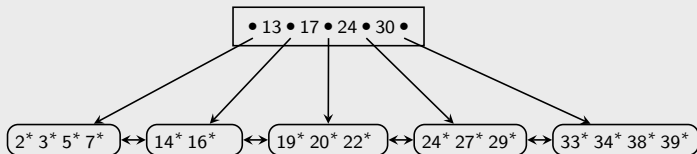
1. Llamar $P = \text{busquedaEnArbol}(k, \text{root})$.
2. Si el espacio libre de P es menor o igual a $2d$, insertar k en P .

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Ejemplo: insertar 15^* en B+-tree (de orden $d = 2$)

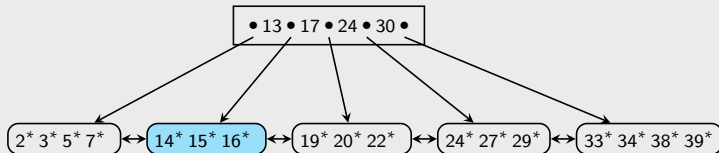


Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Ejemplo: insertar 15^* en B+-tree (de orden $d = 2$)



Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Inserción en B+-tree

Para **insertar** un valor k :

1. Llamar $P = \text{busquedaEnArbol}(k, \text{root})$.
2. Si el espacio libre de P es menor o igual a $2d$, insertar k en P .

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Inserción en B+-tree

Para **insertar** un valor k :

1. Llamar $P = \text{busquedaEnArbol}(k, \text{root})$.
2. Si el espacio libre de P es menor o igual a $2d$, insertar k en P .
3. **En otro caso:** debemos insertar k en P y hacer **split** de $[P + k]!$

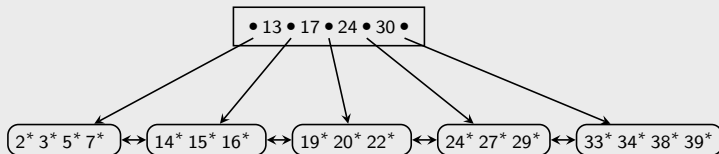
$$\text{donde } [P + k] = k_1^* k_2^* \dots k_{2d}^* k_{2d+1}^*.$$

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Ejemplo: insertar 8^* en B+-tree (de orden $d = 2$)

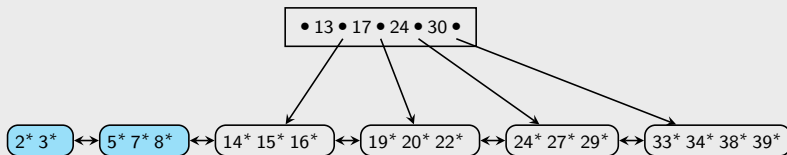


Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Ejemplo: insertar 8^* en B+-tree (de orden $d = 2$)

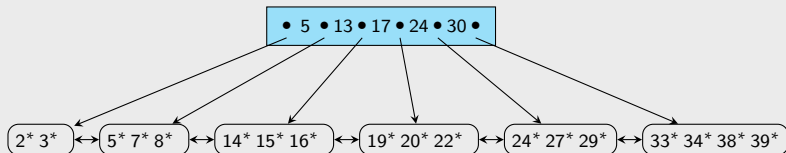


Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Ejemplo: insertar 8^* en B+-tree (de orden $d = 2$)



Insertar un elemento en B+-trees (orden d)

Split de una hoja $[P + k]$ de tamaño $2d + 1$

1. Asuma: $[P + k] = k_1^* k_2^* \dots k_{2d}^* k_{2d+1}^*$ con $k_i < k_{i+1}$.

Insertar un elemento en B+-trees (orden d)

Split de una hoja $[P + k]$ de tamaño $2d + 1$

1. Asuma: $[P + k] = k_1^* k_2^* \dots k_{2d}^* k_{2d+1}^*$ con $k_i < k_{i+1}$.
2. **Divida** $[P + k]$ en dos páginas:

$$P_1 = k_1^* \dots k_d^* \quad \text{y} \quad P_2 = k_{d+1}^* \dots k_{2d+1}^*$$

y reemplaze P por P_1, P_2 en la lista doble-ligada de los datos.

Insertar un elemento en B+-trees (orden d)

Split de una hoja $[P + k]$ de tamaño $2d + 1$

1. Asuma: $[P + k] = k_1^* k_2^* \dots k_{2d}^* k_{2d+1}^*$ con $k_i < k_{i+1}$.
2. **Divida** $[P + k]$ en dos páginas:

$$P_1 = k_1^* \dots k_d^* \quad \text{y} \quad P_2 = k_{d+1}^* \dots k_{2d+1}^*$$

y reemplaze P por P_1, P_2 en la lista doble-ligada de los datos.

3. Seleccione el valor k_{d+1} como **divisor** de P_1 y P_2 .

Insertar un elemento en B+-trees (orden d)

Split de una hoja $[P + k]$ de tamaño $2d + 1$

1. Asuma: $[P + k] = k_1^* k_2^* \dots k_{2d}^* k_{2d+1}^*$ con $k_i < k_{i+1}$.
2. **Divida** $[P + k]$ en dos páginas:

$$P_1 = k_1^* \dots k_d^* \quad \text{y} \quad P_2 = k_{d+1}^* \dots k_{2d+1}^*$$

y reemplaze P por P_1, P_2 en la lista doble-ligada de los datos.

3. Seleccione el valor k_{d+1} como **divisor** de P_1 y P_2 .
4. Reemplace el puntero p en la página P' que apuntaba a la página P por $p_1 k_{d+1} p_2$ donde p_1 apunta a P_1 y p_2 apunta a P_2 .

Insertar un elemento en B+-trees (orden d)

Split de una hoja $[P + k]$ de tamaño $2d + 1$

1. Asuma: $[P + k] = k_1^* k_2^* \dots k_{2d}^* k_{2d+1}^*$ con $k_i < k_{i+1}$.
2. **Divida** $[P + k]$ en dos páginas:

$$P_1 = k_1^* \dots k_d^* \quad \text{y} \quad P_2 = k_{d+1}^* \dots k_{2d+1}^*$$

y reemplaze P por P_1, P_2 en la lista doble-ligada de los datos.

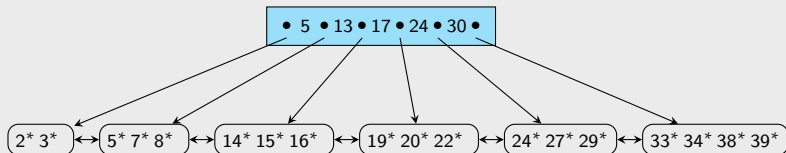
3. Seleccione el valor k_{d+1} como **divisor** de P_1 y P_2 .
4. Reemplace el puntero p en la página P' que apuntaba a la página P por $p_1 k_{d+1} p_2$ donde p_1 apunta a P_1 y p_2 apunta a P_2 .
5. Itere sobre la página de directorio P' que apuntaba a P (**split** de P').

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Ejemplo: insertar 8^* en B+-tree (de orden $d = 2$)

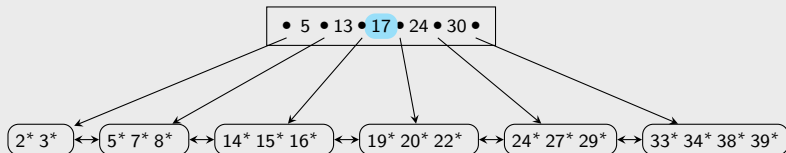


Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Ejemplo: insertar 8^* en B+-tree (de orden $d = 2$)

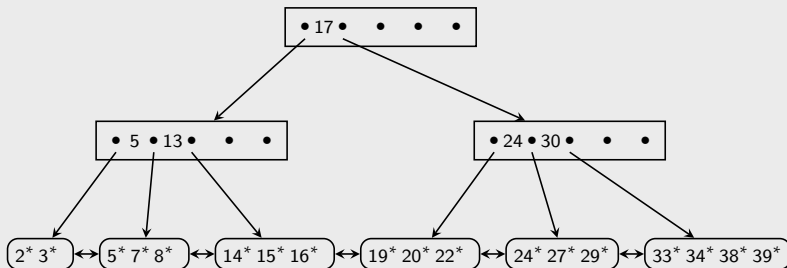


Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Ejemplo: insertar 8^* en B+-tree (de orden $d = 2$)



Insertar un elemento en B+-trees (orden d)

Split de un directorio P de tamaño $2d + 1$

1. Asuma:

$$P = p_0 k_1 p_1 k_2 p_2 \dots p_{2d} k_{2d+1} p_{2d+1}$$

con $k_i < k_{i+1}$.

Insertar un elemento en B+-trees (orden d)

Split de un directorio P de tamaño $2d + 1$

1. Asuma:

$$P = p_0 k_1 p_1 k_2 p_2 \dots p_{2d} k_{2d+1} p_{2d+1}$$

con $k_i < k_{i+1}$.

2. **Divida** P en dos páginas:

$$P_1 = p_0 k_1 \dots k_d p_d \quad \text{y} \quad P_2 = p_{d+1} k_{d+2} \dots k_{2d+1} p_{2d+1}$$

y reemplaze P por P_1, P_2 en el directorio.

Insertar un elemento en B+-trees (orden d)

Split de un directorio P de tamaño $2d + 1$

1. Asuma:

$$P = p_0 k_1 p_1 k_2 p_2 \dots p_{2d} k_{2d+1} p_{2d+1}$$

con $k_i < k_{i+1}$.

2. **Divida** P en dos páginas:

$$P_1 = p_0 k_1 \dots k_d p_d \quad \text{y} \quad P_2 = p_{d+1} k_{d+2} \dots k_{2d+1} p_{2d+1}$$

y reemplaze P por P_1, P_2 en el directorio.

3. Seleccione el valor k_{d+1} como **divisor** de P_1 y P_2 .

Insertar un elemento en B+-trees (orden d)

Split de un directorio P de tamaño $2d + 1$

1. Asuma:

$$P = p_0 k_1 p_1 k_2 p_2 \dots p_{2d} k_{2d+1} p_{2d+1}$$

con $k_i < k_{i+1}$.

2. **Divida** P en dos páginas:

$$P_1 = p_0 k_1 \dots k_d p_d \quad \text{y} \quad P_2 = p_{d+1} k_{d+2} \dots k_{2d+1} p_{2d+1}$$

y reemplaze P por P_1, P_2 en el directorio.

3. Seleccione el valor k_{d+1} como **divisor** de P_1 y P_2 .

4. Reemplace el puntero p en la página P' que apuntaba a la página P por $p_1 k_{d+1} p_2$ donde p_1 apunta a P_1 y p_2 apunta a P_2 .

Insertar un elemento en B+-trees (orden d)

Split de un directorio P de tamaño $2d + 1$

1. Asuma:

$$P = p_0 k_1 p_1 k_2 p_2 \dots p_{2d} k_{2d+1} p_{2d+1}$$

con $k_i < k_{i+1}$.

2. **Divida** P en dos páginas:

$$P_1 = p_0 k_1 \dots k_d p_d \quad \text{y} \quad P_2 = p_{d+1} k_{d+2} \dots k_{2d+1} p_{2d+1}$$

y reemplaze P por P_1, P_2 en el directorio.

3. Seleccione el valor k_{d+1} como **divisor** de P_1 y P_2 .

4. Reemplace el puntero p en la página P' que apuntaba a la página P por $p_1 k_{d+1} p_2$ donde p_1 apunta a P_1 y p_2 apunta a P_2 .

5. Itere sobre la página de directorio P' que apuntaba a P (split de P').

Split del nodo raíz de un B+-tree

- Split de los nodos parte desde las hojas y continua hasta la raíz.

Split del nodo raíz de un B+-tree

- Split de los nodos parte desde las hojas y continua hasta la raíz.
- Si:
 - es necesario hacer split de todos los nodos y
 - el nodo raíz esta lleno,entonces será necesario crear un nuevo nodo raíz.

Split del nodo raíz de un B+-tree

- Split de los nodos parte desde las hojas y continua hasta la raíz.
- Si:
 - es necesario hacer split de todos los nodos y
 - el nodo raíz esta lleno,entonces será necesario crear un nuevo nodo raíz.

Recordar: El nodo raíz es el único que se le permite estar a menos del 50%.

Split del nodo raíz de un B+-tree

- Split de los nodos parte desde las hojas y continua hasta la raíz.
- Si:
 - es necesario hacer split de todos los nodos y
 - el nodo raíz esta lleno,

entonces será necesario crear un nuevo nodo raíz.

Recordar: El nodo raíz es el único que se le permite estar a menos del 50%.

¿qué ocurre con la altura del árbol al hacer split de la raíz?

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Tiempo inserción en B+tree

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Tiempo inserción en B+tree

$$\begin{array}{l} \text{tiempo de} \\ \text{una inserción} \end{array} = \text{altura B+tree} = \mathcal{O}(\log_B(n))$$

Insertar un elemento en B+-trees (orden d)

B+-tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Tiempo inserción en B+tree

$$\begin{array}{l} \text{tiempo de} \\ \text{una inserción} \end{array} = \text{altura B+tree} = \mathcal{O}(\log_B(n))$$

Mismo tiempo que buscar un dato

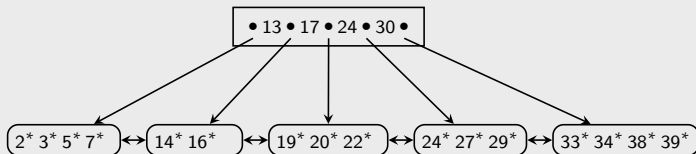
Alternativa a split: redistribución de elementos

*Es posible evitar el crecimiento del árbol
usando redistribución en los nodos vecinos.*

Alternativa a split: redistribución de elementos

*Es posible evitar el crecimiento del árbol
usando redistribución en los nodos vecinos.*

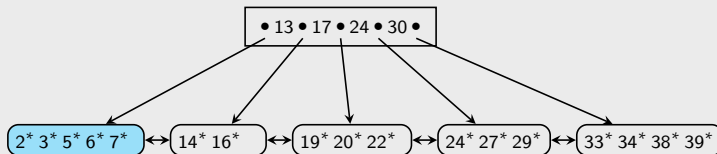
Ejemplo: insertar 6^* en B+-tree (de orden $d = 2$)



Alternativa a split: redistribución de elementos

*Es posible evitar el crecimiento del árbol
usando redistribución en los nodos vecinos.*

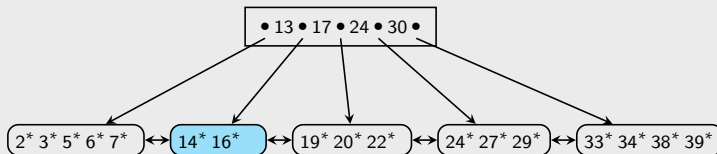
Ejemplo: insertar 6^* en B+-tree (de orden $d = 2$)



Alternativa a split: redistribución de elementos

*Es posible evitar el crecimiento del árbol
usando redistribución en los nodos vecinos.*

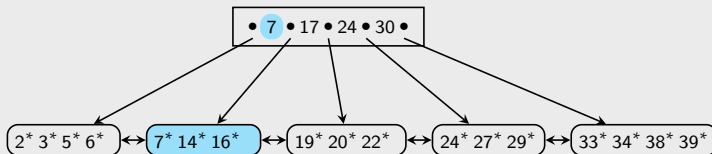
Ejemplo: insertar 6^* en B+-tree (de orden $d = 2$)



Alternativa a split: redistribución de elementos

Es posible evitar el crecimiento del árbol usando redistribución en los nodos vecinos.

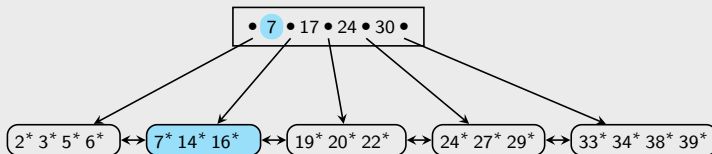
Ejemplo: insertar 6^* en B+-tree (de orden $d = 2$)



Alternativa a split: redistribución de elementos

Es posible evitar el crecimiento del árbol usando redistribución en los nodos vecinos.

Ejemplo: insertar 6^* en B+-tree (de orden $d = 2$)



Redistribución es posible también a **nivel de directorio**.

Outline

B+trees

Inserción en B+trees

Eliminación en B+trees

Eliminar un elemento en B+trees

Eliminar un elemento en B+trees

Para **eliminar** un valor k :

1. Llamar $P = \text{busquedaEnArbol}(k, \text{root})$.

Eliminar un elemento en B+trees

Para **eliminar** un valor k :

1. Llamar $P = \text{busquedaEnArbol}(k, \text{root})$.
2. Si el espacio usado en P es mayor o igual a $d + 1$, eliminar k en P .

Eliminar un elemento en B+trees

Para **eliminar** un valor k :

1. Llamar $P = \text{busquedaEnArbol}(k, \text{root})$.
2. Si el espacio usado en P es mayor o igual a $d + 1$, eliminar k en P .
3. En otro caso: debemos eliminar k en P y **rebalancear** $[P - k]!$

Eliminar un elemento en B+trees

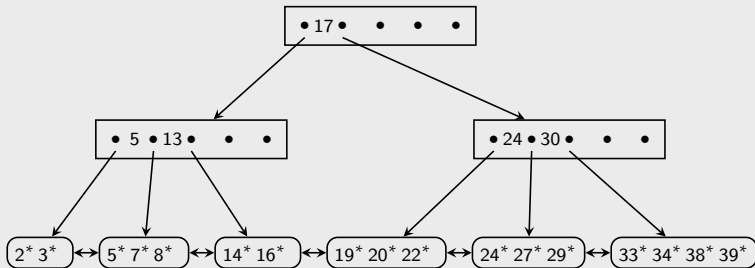
Para **eliminar** un valor k :

1. Llamar $P = \text{busquedaEnArbol}(k, \text{root})$.
2. Si el espacio usado en P es mayor o igual a $d + 1$, eliminar k en P .
3. En otro caso: debemos eliminar k en P y **rebalancear** $[P - k]!$

Veremos el proceso de eliminación en base a ejemplos

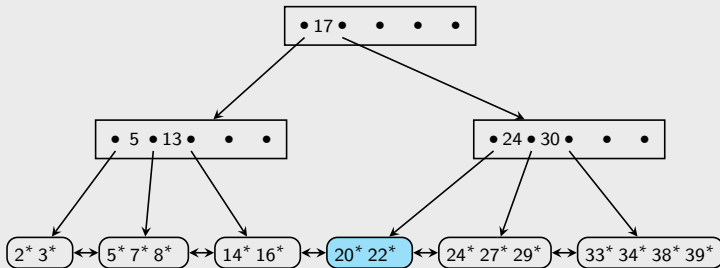
Eliminar un elemento en B+trees: hojas

Ejemplo: eliminar 19^* (orden $d = 2$)



Eliminar un elemento en B+trees: hojas

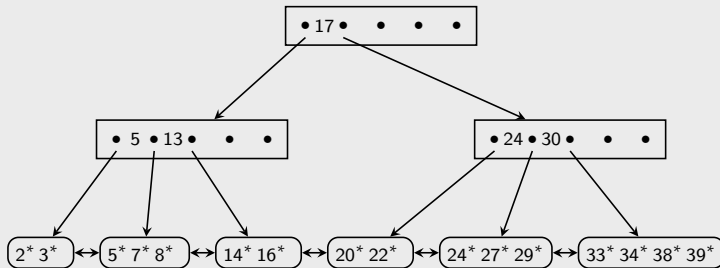
Ejemplo: eliminar 19^* (orden $d = 2$)



Podemos eliminar 19^* ($\#$ -datos $\geq d$)

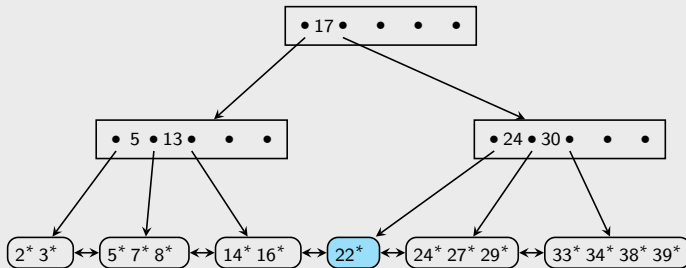
Eliminar un elemento en B+trees: rebalancear hoja

Ejemplo: eliminar 20^* (orden $d = 2$)



Eliminar un elemento en B+trees: rebalancear hoja

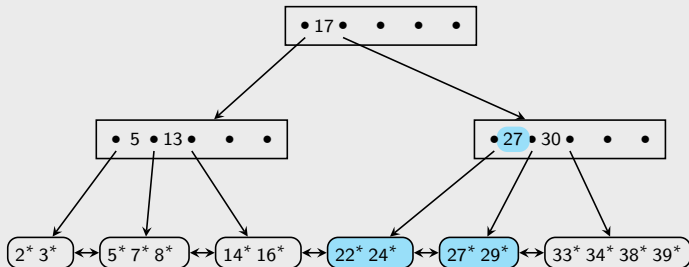
Ejemplo: eliminar 20^* (orden $d = 2$)



Debemos hacer redistribución o **unsplit** de las páginas

Eliminar un elemento en B+trees: rebalancear hoja

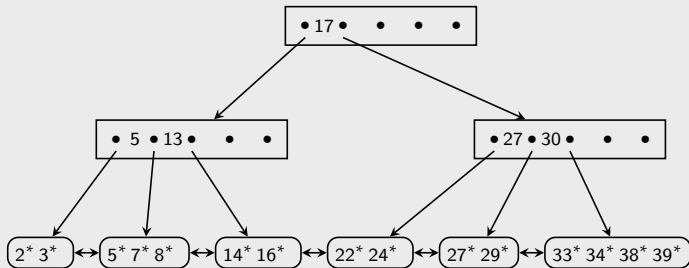
Ejemplo: eliminar 20^* (orden $d = 2$)



Debemos hacer redistribución o **unsplit** de las páginas

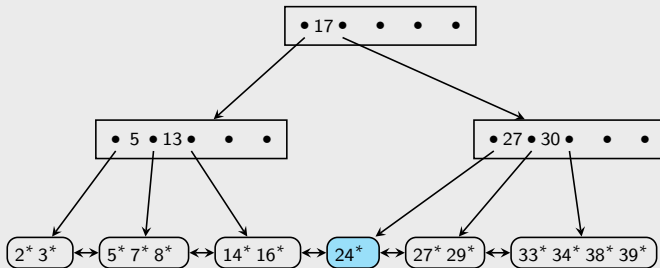
Eliminar un elemento en B+trees: unsplit

Ejemplo: eliminar 22^* (orden $d = 2$)



Eliminar un elemento en B+trees: unsplit

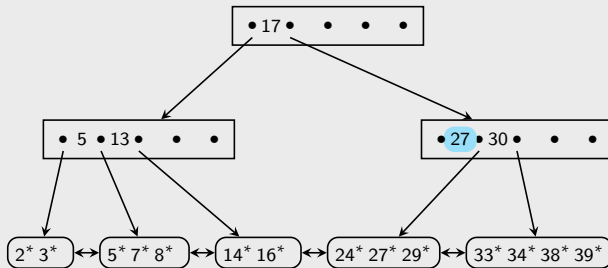
Ejemplo: eliminar 22^* (orden $d = 2$)



Necesitamos hacer un **unsplit** de las páginas

Eliminar un elemento en B+trees: unsplit

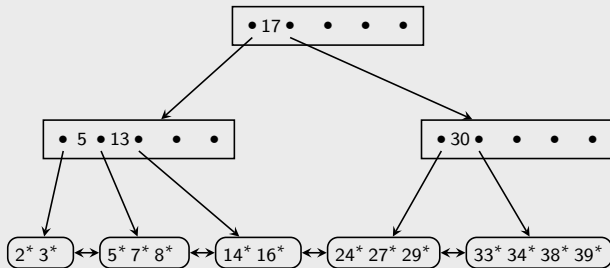
Ejemplo: eliminar 22^* (orden $d = 2$)



Necesitamos hacer un **unsplit** de las páginas

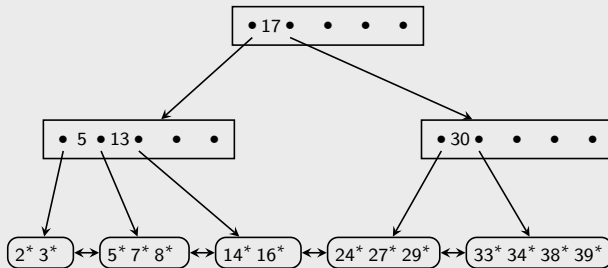
Eliminar un elemento en B+trees: unsplit

Ejemplo: eliminar 22^* (orden $d = 2$)



Eliminar un elemento en B+trees: unsplit

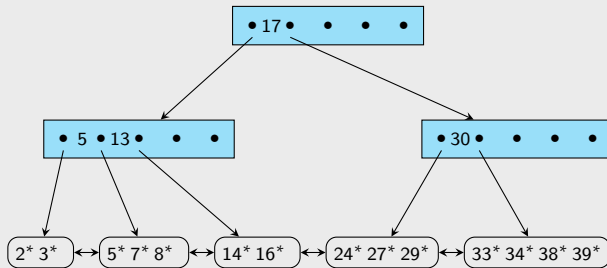
Ejemplo: eliminar 22^* (orden $d = 2$)



Otro **unsplit**, pero ahora del directorio

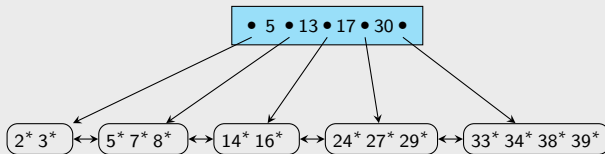
Eliminar un elemento en B+trees: unsplit

Ejemplo: eliminar 22^* (orden $d = 2$)



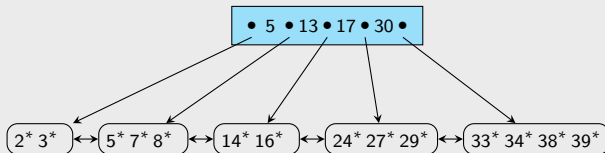
Eliminar un elemento en B+trees: unsplit

Ejemplo: eliminar 22^* (orden $d = 2$)



Eliminar un elemento en B+trees: unsplit

Ejemplo: eliminar 22^* (orden $d = 2$)



Si la **raíz** del árbol tiene **un solo elemento**,
esta es la única manera de **decrementar** la altura del árbol

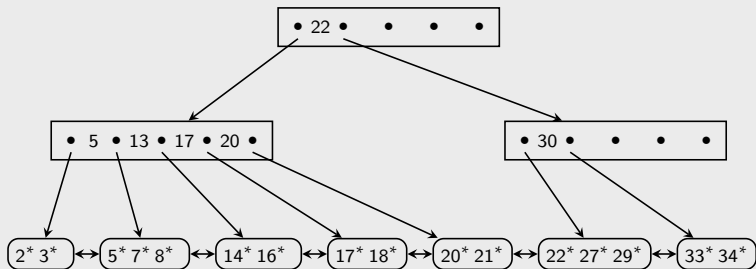
Eliminación y redistribución de elementos

En algunos casos es necesario redistribuir los elementos de un directorio en una eliminación.

Eliminación y redistribución de elementos

En algunos casos es necesario **redistribuir** los elementos de un directorio en una **eliminación**.

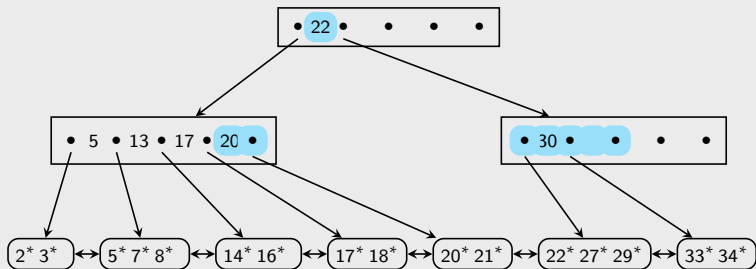
Ejemplo de redistribución de directorio



Eliminación y redistribución de elementos

En algunos casos es necesario **redistribuir** los elementos de un directorio en una **eliminación**.

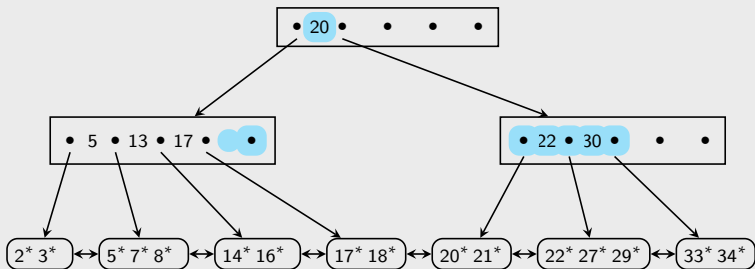
Ejemplo de redistribución de directorio



Eliminación y redistribución de elementos

*En algunos casos es necesario **redistribuir** los elementos de un directorio en una **eliminación**.*

Ejemplo de redistribución de directorio



Eliminar un elemento en B+trees

B+tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Eliminar un elemento en B+trees

B+tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Tiempo eliminación en B+tree

**tiempo de
una eliminación** =

Eliminar un elemento en B+trees

B+tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Tiempo eliminación en B+tree

$$\text{tiempo de una eliminación} = \text{altura B+tree} = \mathcal{O}(\log_B(n))$$

Eliminar un elemento en B+trees

B+tree debe siempre cumplir

1. Regla de altura de hijos.
2. Regla de ocupación mínima.

Tiempo eliminación en B+tree

$$\begin{array}{l} \text{tiempo de} \\ \text{una eliminación} \end{array} = \text{altura B+tree} = \mathcal{O}(\log_B(n))$$

Mismo tiempo que buscar un dato